LICEO LA PAZ

Sistemas de Aprendizaje Automático

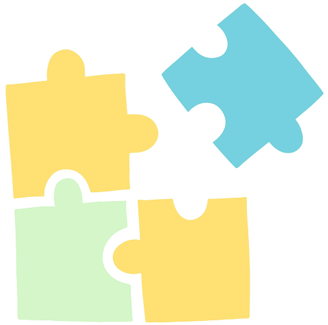
Introducción a Python

Curso 2026 / 2027

Hugo Díaz Devesa

Introducción a Python

- Preparando el entorno con Visual Studio Code
- Creando nuestro entorno virtual
- Seleccionando el intérprete
- Nuestro primer programa en Python
- ¿Qué es una librería?
- Variables y tipos de datos
- Operadores
- Condicionales
- Bucles
- Listas y diccionarios
- Funciones
- Dependencias: requirements.txt
- Scripts .py frente a notebooks .ipynb
- Primer notebook en VS Code
- Python aplicado a datos



Python: qué es y para qué lo vamos a usar

Python es uno de los lenguajes de programación más utilizados actualmente, especialmente en ámbitos relacionados con el análisis de datos, Big Data, inteligencia artificial y automatización.

- Una de sus principales características es su sintaxis sencilla y legible, lo que facilita tanto el aprendizaje como el desarrollo de programas.
- Python es un lenguaje de propósito general, por lo que puede utilizarse para trabajar con datos, desarrollar aplicaciones o automatizar tareas.
- Su uso está muy extendido en análisis de datos gracias a la gran cantidad de librerías disponibles.
- También permite conectarse con archivos, bases de datos, APIs y diferentes fuentes de información.
- En Big Data e inteligencia artificial se utiliza como lenguaje base para aplicar herramientas y librerías especializadas.
- El objetivo inicial será adquirir una base suficiente de Python para poder utilizarlo posteriormente en tareas de tratamiento, análisis y procesamiento de datos.

Origen y evolución de Python

Python fue creado por Guido van Rossum a finales de los años 80 y publicado por primera vez en 1991. Desde entonces, ha evolucionado hasta convertirse en uno de los lenguajes de programación más utilizados, gracias a su sencillez y al apoyo de una amplia comunidad internacional.

- Python es un lenguaje de código abierto, por lo que cualquier persona puede utilizarlo gratuitamente, estudiar su funcionamiento y contribuir a su desarrollo.
- Su evolución está impulsada por una comunidad internacional de desarrolladores, empresas, investigadores y organizaciones.
- La Python Software Foundation (PSF) es una organización sin ánimo de lucro que apoya el desarrollo, la difusión y el ecosistema de Python.
- Las mejoras del lenguaje se proponen y documentan mediante las PEP (Python Enhancement Proposals), que permiten debatir y formalizar cambios.
- La PEP 8 establece recomendaciones para escribir código de forma clara, legible y consistente.
- Actualmente, las decisiones importantes sobre la evolución del lenguaje están supervisadas por un comité de dirección (Steering Council), elegido por desarrolladores principales de Python.

Certificaciones Python

Python no tiene una certificación oficial emitida por la Python Software Foundation (PSF). La referencia más conocida y estructurada es OpenEDG Python Institute, que ofrece un itinerario de certificaciones por niveles.

- PCEP (Inicial) – Variables, operadores, condicionales, bucles y funciones
- PCAP (Intermedio) – Programación orientada a objetos, módulos y excepciones
- PCPP1 (Profesional) – PEP 8, programación avanzada, redes y archivos
- PCPP2 (Profesional avanzado) – Certificación todavía en desarrollo
- PCED (Especialidad) – Análisis de datos con Python, nivel inicial
- PCAD (Especialidad) – Análisis de datos con Python, nivel asociado
- PCEI (Especialidad) – Inteligencia artificial con Python, nivel inicial

Preparando el entorno con Visual Studio Code

Visual Studio Code es un editor de código ligero y flexible que utilizaremos como entorno principal para desarrollar y ejecutar nuestros programas en Python.

- Permite trabajar con archivos y proyectos de forma organizada, facilitando la edición, ejecución y mantenimiento del código.
- Mediante extensiones podemos añadir soporte específico para Python, notebooks y otras herramientas que utilizaremos durante el curso.
- VS Code integra una terminal desde la que podremos ejecutar programas, instalar librerías y gestionar los distintos entornos de trabajo.
- También permite seleccionar fácilmente el intérprete de Python que queremos utilizar en cada proyecto.
- Su integración con Jupyter facilita posteriormente el trabajo con notebooks, muy habituales en análisis de datos, Big Data e inteligencia artificial.
- El objetivo será disponer de un entorno sencillo y común desde el que desarrollar todas las prácticas del curso.

Preparando el entorno con Visual Studio Code

Extensiones recomendadas para trabajar con Python. Se instalan desde la vista de extensiones (Ctrl+Shift+X) o desde la terminal con code --install-extension <id>.

- Python (Microsoft) : Ejecutar, depurar y elegir intérprete. Instala también Pylance (autocompletado y análisis), Python Debugger y Python Environments
- Jupyter (Microsoft) : Abrir y ejecutar .ipynb dentro de VS Code. Incluye renderers y el keymap de Jupyter
- Ruff : Linter y formateador muy rápido (sustituye a flake8, isort y black)
- Data Wrangler : Ver y limpiar DataFrames de pandas de forma visual
- Error Lens : Muestra los errores en la misma línea del código
- Spanish Language Pack : Interfaz en español (opcional)

Instalar Python

VS Code no trae Python, así que hay que instalar el intérprete aparte. Conviene que todos useis la misma versión menor (3.13 o superior).

- Descárgalo de python.org/downloads o instálalo desde PowerShell con winget install Python.Python.3.13.
- En el instalador, marca Add python.exe to PATH.
- Desactiva los alias de la Microsoft Store: Configuración → Aplicaciones → Configuración avanzada → Alias de ejecución de aplicaciones, y apaga python.exe y python3.exe.
- Comprueba que funciona con py --version y py -0 (este último lista las versiones instaladas).

Nuestro primer programa en Python

El primer paso para comenzar a trabajar con Python consiste en crear un archivo de código, escribir una instrucción sencilla y ejecutarla desde nuestro entorno de desarrollo.

- Los programas de Python se guardan normalmente en archivos con extensión .py.
- Una de las primeras instrucciones que utilizaremos será print(), que permite mostrar información por pantalla.
- El código puede ejecutarse directamente desde Visual Studio Code utilizando el intérprete de Python seleccionado.
- También podemos ejecutar nuestros programas desde la terminal mediante el comando python nombre_archivo.py.
- Este primer ejercicio nos permitirá comprobar que Python y Visual Studio Code están correctamente configurados.
- A partir de aquí iremos incorporando progresivamente variables, estructuras de control, funciones y otras herramientas del lenguaje

TASK



Crea una carpeta curso-python, ábrela con code . y crea hola.py:

```
import sys
print(f"Hola desde Python {sys.version}")
print(sys.executable)
```

- Ejecútalo con el botón ▶ de arriba a la derecha.
- Abre la paleta, ejecuta Python: Select Interpreter y comprueba que aparece el Python que instalaste.

Seleccionando el intérprete

El intérprete de Python es el programa encargado de ejecutar nuestro código, y Visual Studio Code nos permite elegir qué instalación de Python utilizar en cada proyecto.

- Un mismo equipo puede tener instaladas varias versiones de Python.
- Seleccionar el intérprete correcto garantiza que el proyecto utilice la versión y las librerías adecuadas.
- En VS Code podemos elegirlo fácilmente desde la paleta de comandos o desde la barra inferior del editor.
- Cuando trabajemos con entornos virtuales, deberemos seleccionar el intérprete correspondiente al entorno creado para ese proyecto.
- Esto evita conflictos entre proyectos que puedan utilizar versiones diferentes de Python o de una misma librería.
- El objetivo es que cada proyecto disponga de un entorno de ejecución independiente y correctamente configurado.

TASK



Abre la carpeta curso-python en VS Code y abre la terminal integrada (Ctrl+ñ).

Comprueba qué Python encuentra el sistema:

- where python y py -0

Modifica hola.py:

```
import sys
print("Versión:", sys.version)
print("Ejecutable:", sys.executable)
```

Ejecútalo con el botón ▶ y anota la ruta del ejecutable.
Pulsa Ctrl+Shift+P → Python: Select Interpreter y revisa qué intérpretes detecta VS Code.
Fíjate en la barra inferior: ahí aparece la versión seleccionada.

¿Coincide la ruta de sys.executable con alguna de las que mostró la terminal en el paso 2?

Creando nuestro entorno virtual

Un entorno virtual permite crear un espacio aislado para cada proyecto de Python, evitando que las librerías y sus versiones interfieran con otros proyectos.

- Cada entorno virtual dispone de su propia configuración y de las librerías instaladas específicamente para ese proyecto.
- Python incorpora la herramienta venv, que permite crear estos entornos de forma sencilla.
- Habitualmente crearemos el entorno dentro de la carpeta del proyecto, utilizando nombres como .venv o venv.
- Una vez creado, debemos activarlo antes de instalar librerías o ejecutar determinados comandos.
- Visual Studio Code puede detectar estos entornos y utilizar automáticamente su intérprete de Python.
- Trabajar con entornos virtuales facilita mantener los proyectos organizados, reproducibles y con sus dependencias correctamente controladas.

TASK



En la terminal de VS Code, dentro de curso-python:

- python -m venv .venv

Explora la carpeta .venv que acaba de aparecer: Scripts/ o bin/, y Lib/site-packages.

Activa el entorno:

- .venv\Scripts\activate

Observa que el prompt ahora empieza por (.venv).

Ejecuta de nuevo hola.py desde la terminal con python hola.py.

¿Qué ruta muestra ahora sys.executable? ¿Es la misma que en la práctica anterior?

¿Qué es una librería?

Una librería es un conjunto de funciones y herramientas desarrolladas previamente que podemos incorporar a nuestros programas para realizar determinadas tareas sin tener que programarlas desde cero.

- Python dispone de una gran cantidad de librerías que amplían las funcionalidades disponibles en el lenguaje.
- Algunas librerías vienen incluidas con Python, mientras que otras deben instalarse de forma adicional.
- Para utilizar una librería en nuestro programa debemos importarla mediante la instrucción import.
- Las librerías permiten reutilizar código probado y facilitan el desarrollo de aplicaciones más complejas.
- En análisis de datos utilizaremos librerías como NumPy, Pandas, Matplotlib o Scikit-learn.
- A lo largo del curso iremos incorporando diferentes librerías según las necesidades de cada tarea o proyecto

TASK



Con el entorno activo, instala una librería:

- python -m pip install requests
- python -m pip list

Añade import requests a hola.py y ejecútalo: funciona.

Desactiva el entorno con deactivate y vuelve a ejecutar python hola.py.

Resultado esperado: ModuleNotFoundError, porque requests solo existe dentro del .venv.

En VS Code, ejecuta Python: Select Interpreter, elige el de .venv y pulsa ▶. Vuelve a funcionar.

¿Por qué el botón ▶ funciona aunque la terminal no tenga el entorno activado?

Dependencias: requirements.txt

Las dependencias son las librerías externas que necesita un proyecto para funcionar correctamente, y pueden registrarse en un archivo llamado requirements.txt.

- Este archivo contiene normalmente el nombre de las librerías utilizadas y, cuando es necesario, su versión concreta.
- Permite conocer rápidamente qué paquetes necesita un proyecto sin tener que instalarlos manualmente uno a uno.
- Podemos generar el archivo con las librerías instaladas utilizando el comando pip freeze > requirements.txt.
- También podemos instalar todas las dependencias de un proyecto mediante pip install -r requirements.txt.
- Esto facilita compartir proyectos entre distintos equipos o reproducir el mismo entorno de trabajo. Lo que se comparte es el requirements.txt, nunca el .venv.
- Su uso ayuda a mantener los proyectos organizados y a evitar problemas provocados por diferencias entre versiones de librerías.

TASK



Activa el entorno de la práctica anterior. El prompt debe mostrar (.venv).

Instala una segunda librería:

- python -m pip install rich

Genera el archivo:

- python -m pip freeze > requirements.txt

Ábrelo en VS Code y revisa su contenido.

Solo hemos instalado 2 librerías. ¿Por qué aparecen más? ¿Qué significa el == seguido de un número?

Desactiva el entorno y borra la carpeta .venv por completo. Ejecuta hola.py: falla con ModuleNotFoundError.

Crea de nuevo el entorno y actívalo. Instala todas las dependencias de una vez:

- python -m pip install -r requirements.txt

Scripts .py frente a notebooks .ipynb

Python puede utilizarse tanto mediante archivos de código tradicionales como a través de notebooks interactivos, dependiendo del tipo de trabajo que queramos realizar.

- Los archivos .py contienen código Python que normalmente se ejecuta de forma secuencial desde el principio hasta el final.
- Los notebooks .ipynb organizan el trabajo en celdas que pueden ejecutarse de forma independiente.
- Los notebooks permiten combinar código, resultados, texto explicativo y visualizaciones dentro de un mismo documento.
- Los scripts resultan especialmente adecuados para programas, automatizaciones y procesos que deben ejecutarse de forma completa y repetible.
- Los notebooks son muy utilizados en análisis de datos, experimentación, Big Data e inteligencia artificial.
- A lo largo del curso utilizaremos ambos formatos según las necesidades de cada práctica y del tipo de trabajo realizado.

Primer notebook en VS Code

Los notebooks permiten trabajar con Python de forma interactiva, ejecutando pequeñas partes del código y observando inmediatamente sus resultados.

- En Visual Studio Code podemos crear y abrir archivos con extensión .ipynb.
- El contenido se organiza en celdas, que pueden contener código Python o texto explicativo.
- Cada celda de código puede ejecutarse de forma independiente sin necesidad de ejecutar todo el archivo.
- Los resultados aparecen directamente debajo de la celda, facilitando la exploración y comprobación de datos.
- Para ejecutar el notebook debemos seleccionar el intérprete o entorno de Python adecuado.
- Este formato será especialmente útil durante el curso para realizar análisis, pruebas, visualizaciones y experimentos con datos.

TASK



Con el .venv activado, instala el kernel de Jupyter:

- python -m pip install ipykernel

Crea prueba.ipynb en VS Code.

Arriba a la derecha, pulsa Select Kernel → Python Environments y elige .venv.

Primera celda (código):

- import sys
- sys.executable

Ejecútala con Shift+Enter. La ruta debe apuntar a tu .venv.
Añade una celda de Markdown con un título: # Mi primer notebook.
Añade otra celda de código con lo mismo que hacía hola.py, usando requests o rich.

En la celda 4 no hemos escrito print. ¿Por qué se muestra el resultado?

TASK



Crea tres celdas de código:

- Celda A => x = 10
- Celda B => x = x * 2
- Celda C => print(x)

Ejecuta A, B y C en orden. Anota el resultado.

Ejecuta B dos veces más y luego C. ¿Qué valor sale ahora?

Fíjate en los números entre corchetes [ ] a la izquierda de cada celda.

Pulsa Restart y luego Run All. ¿Qué resultado sale?

Abre prueba.ipynb con Clic derecho → Open With → Text Editor. ¿Qué tipo de archivo es en realidad?

Sintaxis básica

Python destaca por una sintaxis limpia y legible, diseñada para que el código sea fácil de escribir, leer y mantener.

- La indentación forma parte de la sintaxis: los bloques de código se delimitan con sangría en lugar de llaves.
- No es necesario terminar las instrucciones con punto y coma; cada línea suele contener una instrucción.
- Los comentarios se escriben con #, y las cadenas con triple comilla (docstrings) se usan para documentar funciones, clases y módulos.
- La función print() muestra información por pantalla, y las f-strings permiten insertar variables dentro del texto de forma sencilla.
- Python distingue entre mayúsculas y minúsculas, por lo que nombre y Nombre son identificadores diferentes.
- La guía de estilo PEP 8 define convenciones comunes para escribir código Python claro y consistente.

PEP 8

PEP 8 es la guía oficial de estilo de Python. Establece una serie de recomendaciones para escribir código limpio, legible, consistente y fácil de mantener. Algunas de las principales reglas de PEP 8 son:

- Indentación → Usar 4 espacios por nivel
- Variables → Utilizar snake_case
- Funciones → Utilizar snake_case
- Clases → Utilizar PascalCase
- Constantes → Utilizar MAYUSCULAS
- Espacios → Alrededor de operadores como = o +
- Líneas → Preferiblemente un máximo de 79 caracteres
- Importaciones → Situarlas al principio del archivo
- Líneas en blanco → Separar funciones y clases para facilitar la lectura

TASK



Descarga del repositorio del curso el cuaderno 01_sintaxis_basica.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Variables y tipos de datos

Las variables son nombres que hacen referencia a datos en memoria, permitiendo utilizarlos y modificarlos durante la ejecución de un programa.

- En Python no es necesario indicar previamente el tipo de una variable, ya que se determina automáticamente según el valor asignado.
- Los tipos de datos básicos incluyen números enteros int, números decimales float, textos str y valores lógicos bool.
- Una misma variable puede cambiar de valor, e incluso de tipo, a lo largo del programa.
- Los nombres de las variables deben ser claros y descriptivos para facilitar la lectura y mantenimiento del programa.
- Python permite convertir datos de un tipo a otro mediante funciones como int(), float() o str().
- Comprender los tipos de datos es fundamental para trabajar posteriormente con estructuras más complejas y con información procedente de archivos, bases de datos o APIs.

TASK



Descarga del repositorio del curso el cuaderno 02_variables_tipos.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Operadores

Los operadores permiten realizar cálculos, comparaciones y evaluaciones lógicas utilizando los valores almacenados en variables.

- Los operadores aritméticos permiten realizar operaciones como suma +, resta -, multiplicación *, división /, división entera //, resto % o potencia **.
- Los operadores de comparación permiten comprobar relaciones entre valores mediante expresiones como ==, !=, >, <, >= o <=.
- Los operadores lógicos and, or y not permiten combinar o modificar condiciones dentro de un programa.
- El resultado de muchas operaciones de comparación y lógica es un valor booleano: True o False.
- Los operadores pueden combinarse dentro de expresiones más complejas para realizar cálculos o tomar decisiones.
- Su uso será fundamental posteriormente para filtrar datos, validar información y construir condiciones dentro de nuestros programas.

TASK



Descarga del repositorio del curso el cuaderno 03_operadores.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Condicionales

Los condicionales permiten que un programa tome decisiones y ejecute diferentes bloques de código en función de si se cumplen determinadas condiciones.

- La estructura principal se construye con if, que permite comprobar si una condición es verdadera.
- Podemos utilizar elif para evaluar condiciones alternativas cuando la anterior no se cumple.
- La instrucción else permite definir qué debe ocurrir cuando ninguna de las condiciones anteriores es verdadera.
- Las condiciones suelen construirse utilizando operadores de comparación y operadores lógicos.
- En Python la indentación es fundamental, ya que determina qué instrucciones pertenecen a cada bloque condicional.
- Los condicionales serán especialmente útiles para validar datos, clasificarlos o aplicar diferentes acciones según sus valores.
- Python ofrece también la expresión condicional (valor_a if condición else valor_b) y, desde la versión 3.10, la estructura match/case para comparar un valor con varios patrones.

TASK



Descarga del repositorio del curso el cuaderno 04_condicionales.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Bucles

Los bucles permiten repetir un conjunto de instrucciones varias veces, evitando tener que escribir el mismo código de forma repetitiva.

- El bucle for se utiliza principalmente para recorrer elementos como textos, rangos de números o, más adelante, listas y otras colecciones.
- El bucle while permite repetir instrucciones mientras se cumpla una determinada condición.
- En cada repetición podemos acceder al elemento actual y realizar operaciones sobre él.
- Python permite controlar la ejecución de los bucles mediante instrucciones como break y continue.
- Los bucles pueden combinarse con condicionales para aplicar diferentes acciones según los valores procesados.
- Su uso será fundamental para recorrer, transformar y analizar conjuntos de datos de forma automatizada.

TASK



Descarga del repositorio del curso el cuaderno 05_bucles.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Colecciones

Las listas y los diccionarios permiten almacenar varios valores dentro de una misma estructura, facilitando la organización y el tratamiento de conjuntos de datos.

- Las listas permiten guardar elementos ordenados y acceder a ellos mediante su posición o índice.
- Una lista puede contener números, textos, valores lógicos e incluso otras estructuras de datos.
- Los diccionarios almacenan información mediante pares clave-valor, permitiendo identificar cada dato mediante un nombre.
- Podemos añadir, modificar, eliminar y consultar elementos tanto en listas como en diccionarios.
- Estas estructuras pueden recorrerse fácilmente mediante bucles para procesar todos sus elementos.
- Serán especialmente útiles para trabajar posteriormente con datos procedentes de archivos JSON, APIs y otras fuentes de información.

TASK



Descarga del repositorio del curso el cuaderno 06_colecciones.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Funciones

Las funciones permiten agrupar instrucciones que realizan una tarea concreta, facilitando la reutilización y organización del código.

- En Python las funciones se definen mediante la palabra clave def.
- Una función puede recibir datos de entrada mediante parámetros.
- También puede devolver un resultado utilizando la instrucción return.
- Las funciones permiten evitar repetir el mismo código en distintas partes de un programa.
- Dividir un programa en funciones facilita su lectura, mantenimiento y reutilización.
- Su uso será especialmente útil para organizar procesos de tratamiento, transformación y análisis de datos.
- En Python las funciones son objetos: pueden pasarse como argumento a otras funciones, guardarse en variables o crearse dentro de otras funciones.

TASK



Descarga del repositorio del curso el cuaderno 07_funciones.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Errores y excepciones

Las funciones permiten agrupar instrucciones que realizan una tarea concreta, facilitando la reutilización y organización del código.

- En Python las funciones se definen mediante la palabra clave def.
- Una función puede recibir datos de entrada mediante parámetros.
- También puede devolver un resultado utilizando la instrucción return.
- Las funciones permiten evitar repetir el mismo código en distintas partes de un programa.
- Dividir un programa en funciones facilita su lectura, mantenimiento y reutilización.
- Su uso será especialmente útil para organizar procesos de tratamiento, transformación y análisis de datos.

TASK



Descarga del repositorio del curso el cuaderno 08_excepciones.ipynb.

Cópialo a tu repositorio personal.

Retos: Ejecuta, observa y escribe tu explicación en la celda antes de continuar.

Ejercicios: completa el código y ejecuta la celda de comprobación hasta ver ¡Correcto!

Python aplicado a datos

Python es una de las herramientas más utilizadas para trabajar con datos, ya que permite obtenerlos, transformarlos, analizarlos y representarlos mediante diferentes librerías especializadas.

- Podemos trabajar con información procedente de archivos CSV, JSON, bases de datos, APIs y otras fuentes.
- Librerías como Pandas facilitan la carga, limpieza, transformación y análisis de conjuntos de datos.
- NumPy permite realizar cálculos numéricos de forma eficiente sobre grandes cantidades de información.
- Matplotlib y otras librerías permiten generar gráficos y visualizaciones a partir de los datos procesados.
- Python también se utiliza como base para aplicar técnicas de aprendizaje automático, inteligencia artificial y análisis predictivo.
- A lo largo del curso iremos incorporando estas herramientas de forma progresiva según las necesidades de cada bloque.

TASK



Realiza la tarea final 09_tarea_final_cifras.ipynb.